

FULL STACK | BEFORE DATABASES

Readiness checklist

Week 3 Day 1 to Week 6 Day 4

17 topics

19 official course days | interactive checkboxes | source links

HOW TO USE

- Check a skill only when you can explain it without notes.
- Run the short example or complete one practice task when unsure.
- Open the official source link when a gap needs a full lesson.
- Checkbox state is saved by your PDF reader when the file is saved.

TOPIC MAP

- | | | | |
|----|--|----|-----------------------------|
| 01 | JavaScript foundations | 02 | Functions and DOM basics |
| 03 | DOM events and forms | 04 | DOM mini-project workflow |
| 05 | Advanced functions | 06 | Advanced array methods |
| 07 | Objects, classes, inheritance, and dates | 08 | HTTP and form methods |
| 09 | Browser mini-project integration | 10 | Asynchronous JavaScript |
| 11 | Fetch and async / await | 12 | Async mini-project workflow |
| 13 | TypeScript foundations | 14 | Advanced TypeScript |
| 15 | Hackathon integration | 16 | Node.js fundamentals |
| 17 | Web servers and Express | | |

VALUES TO DECISIONS TO REPETITION

JavaScript foundations

Use JavaScript in a browser. Store values, inspect types, compare safely, choose a path, and repeat work.

I UNDERSTAND THIS

- ☐ I can link a JavaScript file to HTML and use the console.
- ☐ I can explain string, number, boolean, null and undefined.
- ☐ I can choose const or let and use clear variable names.
- ☐ I can use strict comparisons and logical operators.
- ☐ I can write if / else and for / while loops.

SMALL EXAMPLE

```
const score = 82;  
const passed = score >= 70;  
if (passed) console.log('Ready');
```

PRACTICE

- Convert a temperature and print a clear result.
- Loop through five scores and count passes.
- Predict each comparison before running it.

COMMON MISTAKES

- Using == when strict equality === is clearer.
- Changing a const value or forgetting let for reassignment.
- Creating an infinite loop because the counter never changes.

OFFICIAL SOURCES

W3 D1 | Introduction to JavaScript

REUSABLE LOGIC + PAGE UPDATES

Functions and DOM basics

Put repeated logic in functions. Read and change the page through DOM nodes, attributes, styles, and created elements.

I UNDERSTAND THIS

- ☐ I can define, call, and return from a function.
- ☐ I can explain parameters, arguments, and local scope.
- ☐ I can select one or many DOM elements.
- ☐ I can change text, classes, styles, and attributes.
- ☐ I can create, append, and remove DOM nodes.

SMALL EXAMPLE

```
function label(name) { return `Hello ${name}`; }  
document.querySelector('h1').textContent = label('Ada')
```

PRACTICE

- Write a function with parameters and a returned value.
- Select an element and change its text, class, and attribute.
- Create a list item and append it to a list.

COMMON MISTAKES

- Logging a result instead of returning it.
- Calling a function before required data exists.
- Assuming `querySelector` always finds an element.

OFFICIAL SOURCES

W3 D2 | Functions & DOM introduction

USER ACTION TO EVENT TO UPDATE

DOM events and forms

React to clicks, input, submit, and keyboard events. Read form data and understand propagation.

I UNDERSTAND THIS

- ☐ I can register and remove an event listener.
- ☐ I can use event.target and event.currentTarget correctly.
- ☐ I can read and validate form values.
- ☐ I can explain bubbling and use event delegation.
- ☐ I can use timers for a controlled UI update.

SMALL EXAMPLE

```
form.addEventListener('submit', (event) => {  
  event.preventDefault();  
  console.log(new FormData(form));  
});
```

COMMON MISTAKES

- Calling the handler while registering it.
- Forgetting preventDefault on a handled form submit.
- Ignoring bubbling and triggering parent behavior by accident.

OFFICIAL SOURCES

W3 D3 | Learning DOM Events

PRACTICE

- Build a form that adds a card without reloading.
- Use one parent listener for several child buttons.
- Start and stop a small timer-driven animation.

PLAN TO BUILD TO TEST TO REFINE

DOM mini-project workflow

Combine functions, DOM updates, events, and form data in one browser project.

I UNDERSTAND THIS

- ☐ I can break a UI task into small functions.
- ☐ I can keep data and rendered DOM in sync.
- ☐ I can handle empty and invalid user input.
- ☐ I can debug with DevTools and focused console output.
- ☐ I can describe my project without reading the code line by line.

SMALL EXAMPLE

```
state.push(nextItem);  
render(state); // one render path keeps UI and data a
```

PRACTICE

- Split the project into data, render, and event functions.
- Test empty input, repeated actions, and reset behavior.
- Explain the event-to-screen data flow.

COMMON MISTAKES

- Mixing all logic inside one event handler.
- Updating HTML but not the underlying data.
- Testing only the happy path.

OFFICIAL SOURCES

W3 D4 | Mini-Project Day

SCOPE, CALLBACKS, CLOSURES, COMPOSITION

Advanced functions

Treat functions as values. Pass them, return them, preserve lexical state, and build focused reusable behavior.

I UNDERSTAND THIS

- ☐ I can use function declarations, expressions, and arrow functions.
- ☐ I can pass a callback and explain when it runs.
- ☐ I can explain lexical scope and closure.
- ☐ I can use rest parameters and spread arguments.
- ☐ I can recognize currying and partial application.

SMALL EXAMPLE

```
const multiply = (a: number) => (b: number) => a * b;  
const double = multiply(2);
```

PRACTICE

- Write a callback and pass it to another function.
- Create a closure that keeps a private counter.
- Convert a multi-argument function to a curried form.

COMMON MISTAKES

- Confusing a function reference with a function call.
- Expecting var to have block scope.
- Using this inside an arrow function as if it were a method.

OFFICIAL SOURCES

W3 D5 | Advanced JavaScript Functions

TRANSFORM COLLECTIONS CLEARLY

Advanced array methods

Choose an array method from the result you need: side effect, transformed array, subset, one item, boolean, or accumulated value.

I UNDERSTAND THIS

- ☐ I can choose between `forEach`, `map`, `filter`, and `reduce`.
- ☐ I can use `find`, `some`, and `every`.
- ☐ I can write readable callback functions.
- ☐ I can avoid accidental mutation with spread and non-mutating methods.
- ☐ I can chain methods and explain every intermediate result.

SMALL EXAMPLE

```
const total = orders
  .filter(order => order.paid)
  .reduce((sum, order) => sum + order.total, 0);
```

PRACTICE

- Map users to display names.
- Filter active records and find one matching ID.
- Reduce cart items to a total.
-

COMMON MISTAKES

- Forgetting `return` inside a block-bodied callback.
- Using `map` when no transformed array is needed.
- Mutating the original array unexpectedly.

OFFICIAL SOURCES

W4 D1 | Advanced Array methods

MODEL STRUCTURED DATA AND BEHAVIOR

Objects, classes, inheritance, and dates

Read and rebuild objects, use destructuring and shallow copies, model instances with classes, and calculate with Date values.

I UNDERSTAND THIS

- ☐ I can use Object.keys, values, entries, and fromEntries.
- ☐ I can destructure, rename, default, and shallow-clone properties.
- ☐ I can explain object identity and this.
- ☐ I can create classes, instances, methods, getters, and setters.
- ☐ I can use extends and super correctly.
- ☐ I can read dates and calculate a time difference.

SMALL EXAMPLE

```
class Dog extends Animal {  
  constructor(name: string) { super(name); }  
  speak() { return `${this.name} barks`; }  
}
```

COMMON MISTAKES

- Treating a spread copy as a deep copy.
- Using this before super in a child constructor.
- Forgetting that getMonth returns 0-11.

OFFICIAL SOURCES

W4 D2 | Advanced Object Methods

PRACTICE

- Turn Object.entries output back into an object.
- Build a parent class and a child class with super.
- Calculate days between two Date values.

BROWSER REQUEST TO SERVER RESPONSE

HTTP and form methods

Understand request structure and choose GET or POST based on intent. Encode, send, inspect, and validate form data.

I UNDERSTAND THIS

- ☐ I can explain request, response, method, URL, headers, body, and status.
- ☐ I can choose GET for retrieval and POST for submitted data.
- ☐ I can read query parameters and encoded form data.
- ☐ I can inspect a request in the browser Network panel.
- ☐ I can validate input on the client while still expecting server validation.

SMALL EXAMPLE

```
GET /search?q=javascript HTTP/1.1

POST /signup HTTP/1.1
Content-Type: application/json
```

COMMON MISTAKES

- Using GET for a state-changing action.
- Assuming hidden form fields are secure.
- Forgetting that network requests can fail or return non-2xx status.

OFFICIAL SOURCES

W4 D3 | HTTP & Form method GET and POST

PRACTICE

- Inspect request method, URL, headers, and response in DevTools.
- Build equivalent GET and POST forms.
- Explain which data belongs in a URL and which does not.

STATE + DOM + NETWORK THINKING

Browser mini-project integration

Build a small feature by separating data, UI rendering, events, validation, and request boundaries.

I UNDERSTAND THIS

- ☐ I can define a small state model before editing the DOM.
- ☐ I can keep rendering separate from event wiring.
- ☐ I can show clear loading, success, empty, and error states.
- ☐ I can test repeated actions and refresh behavior.
- ☐ I can remove duplication before submission.

SMALL EXAMPLE

```
async function submit(data: FormData) {  
  setBusy(true);  
  try { /* request */ } finally { setBusy(false); }  
}
```

COMMON MISTAKES

- No visible feedback after a user action.
- Duplicate event listeners after re-rendering.
- Treating a console success message as a complete UI.

OFFICIAL SOURCES

W4 D4 | Mini-Project day

PRACTICE

- Draw the user flow before coding.
- Add empty, loading, success, and error states.
- Test the project after a full page reload.

SCHEDULE WORK WITHOUT BLOCKING

Asynchronous JavaScript

Understand the call stack, task queues, timers, callbacks, and Promise states. Control success and failure paths.

I UNDERSTAND THIS

- ☐ I can explain synchronous code, the call stack, and queued work.
- ☐ I can use setTimeout and setInterval safely.
- ☐ I can explain pending, fulfilled, and rejected Promises.
- ☐ I can chain then, catch, and finally.
- ☐ I can predict basic event-loop output order.

SMALL EXAMPLE

```
Promise.resolve('data')
  .then(value => value.toUpperCase())
  .catch(error => console.error(error));
```

PRACTICE

- Predict timer and Promise output order before running it.
- Wrap delayed work in a Promise.
- Chain a transformation and one shared catch.

COMMON MISTAKES

- Expecting a timer delay to be an exact execution time.
- Forgetting to return a Promise from a then callback.
- Leaving a rejected Promise unhandled.

OFFICIAL SOURCES

W4 D5 | Asynchronous JavaScript

REQUEST TO VALIDATE TO PARSE TO USE

Fetch and async / await

Fetch remote data with explicit status checks. Await each asynchronous step and handle failures near the user-facing boundary.

I UNDERSTAND THIS

- ☐ I can send a fetch request and inspect its response.
- ☐ I can check response.ok before parsing JSON.
- ☐ I can write and call an async function.
- ☐ I can use try / catch / finally for UI-safe errors.
- ☐ I can use Promise.all for independent work.

SMALL EXAMPLE

```
const response = await fetch('/api/items');  
if (!response.ok) throw new Error(`HTTP ${response.status} error`);  
const items = await response.json();
```

PRACTICE

- Fetch JSON and render a small list.
- Handle a non-2xx response and invalid data.
- Run independent requests with Promise.all.

COMMON MISTAKES

- Assuming fetch rejects for every HTTP error status.
- Forgetting await before response.json().
- Running independent requests sequentially without a reason.

OFFICIAL SOURCES

W5 D1 | Fetch & Async/Await

RELIABLE REMOTE-DATA UI

Async mini-project workflow

Turn an API task into a predictable interface with explicit states, cancellation awareness, and recoverable errors.

I UNDERSTAND THIS

- ☐ I can model idle, loading, ready, empty, and error states.
- ☐ I can prevent duplicate requests.
- ☐ I can render API data without unsafe HTML injection.
- ☐ I can recover from failure without reloading the page.
- ☐ I can trace one request from event to rendered result.

SMALL EXAMPLE

```
type ViewState<T> =  
  | { kind: 'loading' }  
  | { kind: 'ready'; data: T }  
  | { kind: 'error'; message: string };
```

COMMON MISTAKES

- Rendering stale results from an older request.
- Leaving controls active during duplicate submission.
- Showing raw technical errors to the user.

OFFICIAL SOURCES

W5 D2 | Mini-Project Day - Asynchronous JavaScript

PRACTICE

- Build search with loading and no-results feedback.
- Disable duplicate submission while a request runs.
- Retry after a simulated failure.

MAKE EXPECTED SHAPES EXPLICIT

TypeScript foundations

Use static types to describe values, function contracts, object shapes, and safe alternatives before runtime.

I UNDERSTAND THIS

- ☐ I can explain inference and add an annotation when useful.
- ☐ I can type arrays, tuples, objects, and functions.
- ☐ I can use unions, literals, aliases, and interfaces.
- ☐ I can use optional and readonly properties.
- ☐ I can choose unknown over any and narrow before use.

SMALL EXAMPLE

```
type User = { id: number; name: string; active?: boolean; }  
const label = (user: User): string => user.name;
```

PRACTICE

- Type variables, arrays, object parameters, and return values.
- Replace any with unknown and narrow it safely.
- Create a union for a small UI state.

COMMON MISTAKES

- Adding types everywhere when inference is already clear.
- Using any to silence a useful error.
- Confusing compile-time types with runtime validation.

OFFICIAL SOURCES

W5 D3 | Introduction To TypeScript And Key Concepts

NARROW, REUSE, AND COMPOSE TYPES

Advanced TypeScript

Use generics, guards, utility types, classes, and typed modules to keep larger code precise without duplicating shapes.

I UNDERSTAND THIS

- ☐ I can narrow with typeof, in, instanceof, and custom guards.
- ☐ I can use a discriminated union and exhaustive switch.
- ☐ I can write a useful generic function or interface.
- ☐ I can use Partial, Pick, Omit, Record, and Readonly.
- ☐ I can type a class, DOM element, module, and API result.
- ☐ I can read a compiler error and fix the data contract.

SMALL EXAMPLE

```
function first<T>(items: T[]): T | undefined {  
  return items[0];  
}  
  
const isError = (value: unknown): value is Error => value instanceof Error
```

PRACTICE

- Write a generic function and a generic API response type.
- Narrow a discriminated union with a switch.
- Use Pick or Omit to derive a form type.

COMMON MISTAKES

- Using a type assertion instead of proving the type.
- Making a generic that adds no relationship between values.
- Forgetting exhaustive handling when a union grows.

OFFICIAL SOURCES

[W5 D4 | Advanced TypeScript Concepts and Applications](#)

[W5 D5 | Advanced TypeScript Concepts and Applications](#)

SCOPE SMALL. SHIP A WORKING FLOW.

Hackathon integration

Use the JavaScript, DOM, async, and TypeScript toolkit under a time limit. Prioritize one complete user flow over many unfinished features.

I UNDERSTAND THIS

- ☐ I can reduce an idea to one testable user flow.
- ☐ I can split work into small owned tasks.
- ☐ I can use Git commits and merge without losing working code.
- ☐ I can handle loading, empty, invalid, and failure paths.
- ☐ I can demo the result and explain one important decision.

SMALL EXAMPLE

```
Must have: input to validation to result
Nice to have: polish, extra filters, animation
```

PRACTICE

- Write a must-have list before coding.
- Build a vertical slice that works end to end.
- Prepare a two-minute demo and one technical explanation.

COMMON MISTAKES

- Starting with styling before the main flow works.
- Adding dependencies without checking the cost.
- Merging large changes without testing the shared branch.

OFFICIAL SOURCES

W6 D1 | [Hackathon #1](#)

W6 D2 | [Hackathon #1](#)

JAVASCRIPT OUTSIDE THE BROWSER

Node.js fundamentals

Run JavaScript in Node, use modules and npm, read environment values, and work with files and JSON asynchronously.

I UNDERSTAND THIS

- ☐ I can explain how Node differs from browser JavaScript.
- ☐ I can create package.json and use npm scripts.
- ☐ I can import and export ES modules.
- ☐ I can use process arguments and environment variables.
- ☐ I can read, write, parse, and stringify JSON safely.

SMALL EXAMPLE

```
import { readFile } from 'node:fs/promises';
const text = await readFile('data.json', 'utf8');
const data = JSON.parse(text);
```

PRACTICE

- Create an npm project with a start script.
- Export and import one function.
- Read a JSON file and handle a missing-file error.
-

COMMON MISTAKES

- Using browser globals such as document in Node.
- Mixing CommonJS and ES module syntax without configuration.
- Blocking the event loop with synchronous file work in a server path.

OFFICIAL SOURCES

W6 D3 | Node.js Introduction

ROUTE A REQUEST. RETURN A RESPONSE.

Web servers and Express

Create a Node web server, define Express middleware and routes, validate input, return JSON, and handle missing routes and errors.

I UNDERSTAND THIS

- ☐ I can explain request, middleware, route handler, and response order.
- ☐ I can start an Express server on a configured port.
- ☐ I can build GET and POST routes that return JSON.
- ☐ I can read route params, query values, and request bodies.
- ☐ I can choose a useful status code.
- ☐ I can add 404 and error-handling middleware.

SMALL EXAMPLE

```
app.get('/api/items/:id', (req, res) => {  
  res.json({ id: req.params.id });  
});
```

PRACTICE

- Create GET and POST JSON routes.
- Read params, query values, and a JSON body.
- Return 404 and centralized error responses.

COMMON MISTAKES

- Sending more than one response for one request.
- Forgetting `express.json` before reading `req.body`.
- Trusting request data without validation.

OFFICIAL SOURCES

W6 D4 | Node.js Web Server